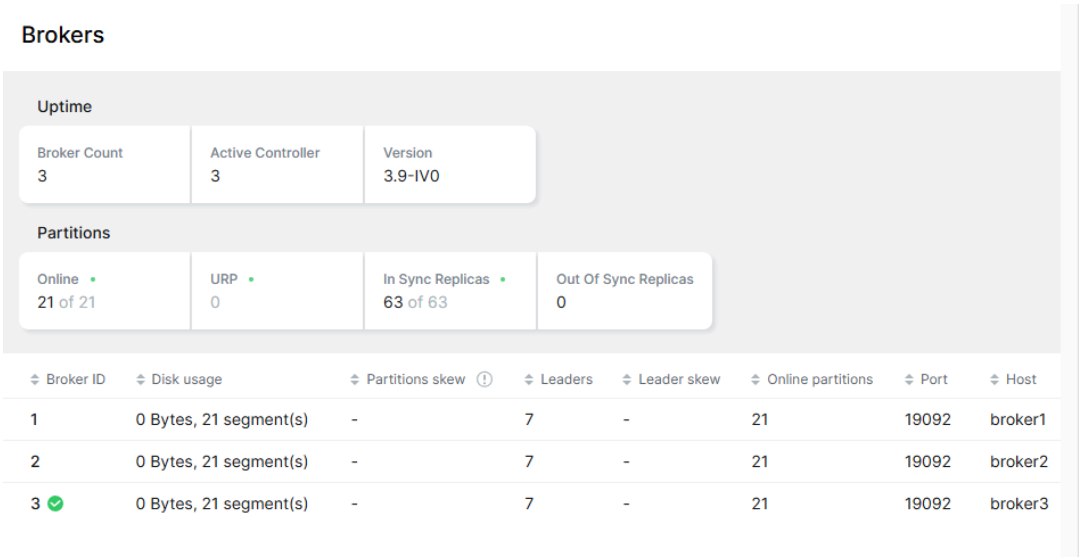
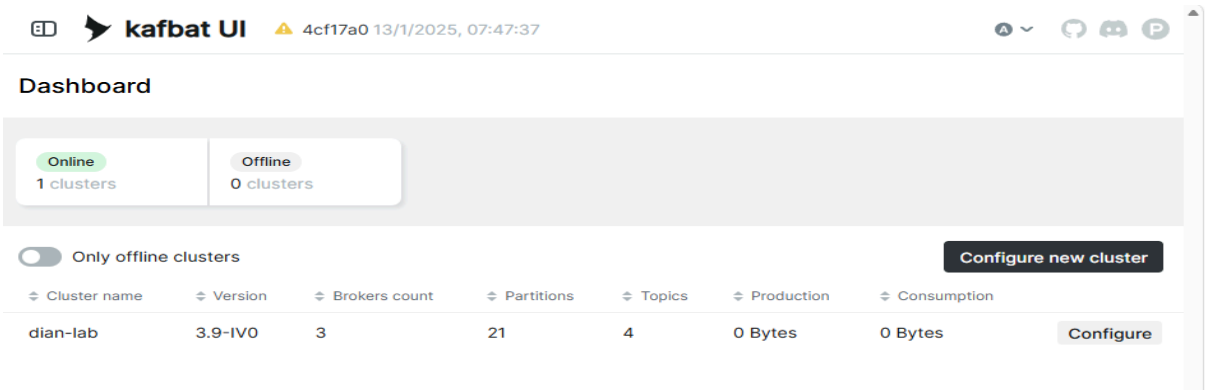
EVIDENCIAS EJECUCIÓN LABORATORIO 1

```
sorjuelav@DESKTOP-7O817GL:/home/equipo6/kafka_env$ docker compose -f docker-compose-lab-kafka.yml up -d
[+] up 31/31
 ✔Image apache/kafka:3.9.0            Pulled                                    16.2s
 ✔Image ghcr.io/kafbat/kafka-ui:v1.1.0 Pulled                                   24.0s
 ✔Network kafka_env_default           Created                                    0.1s
 ✔Volume kafka_env_data1              Created                                    0.0s
 ✔Volume kafka_env_data2              Created                                    0.0s
 ✔Volume kafka_env_data3              Created                                    0.0s
 ✔Container broker3                   Healthy                                   19.9s
 ✔Container broker2                   Healthy                                   20.4s
 ✔Container broker1                   Healthy                                   20.4s
 ✔Container kafka-ui                  Started                                    2.3s
 ✔Container bootstrap-topics          Started                                   19.7s
```

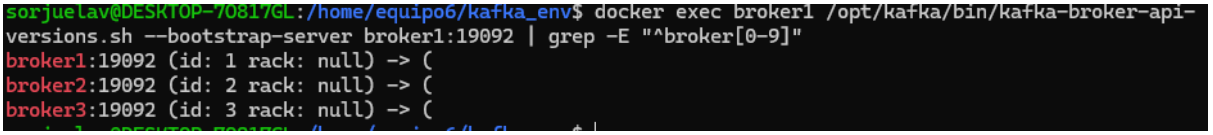


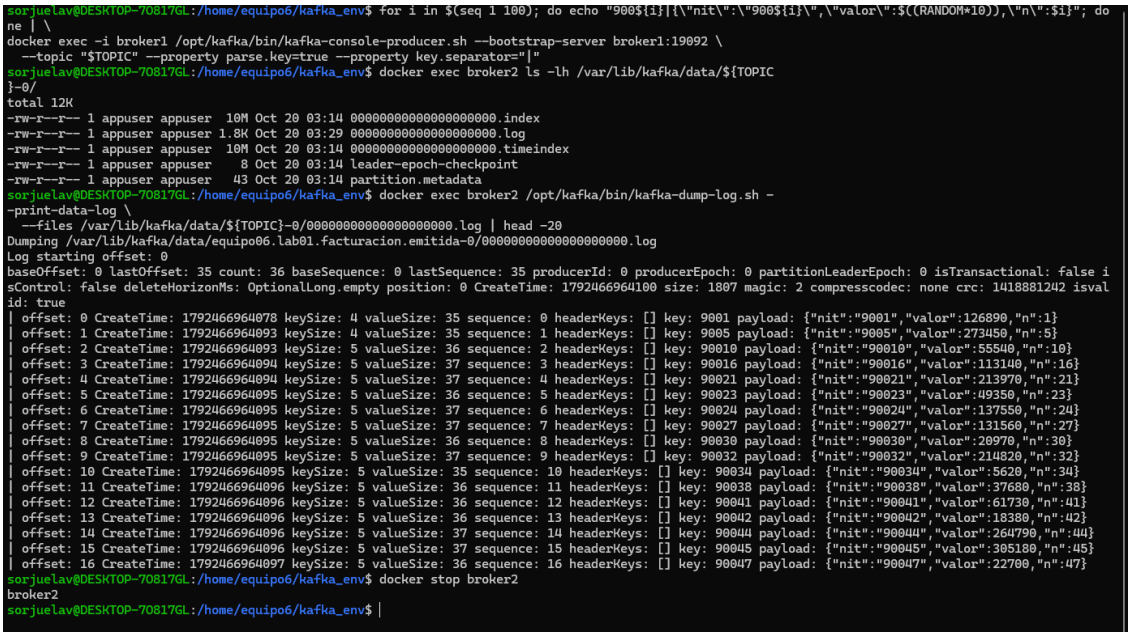



```
sorjuelav@DESKTOP-7O817GL:/home/equipo6/kafka_env$ docker compose -f docker-compose-lab-kafka.yml ps
NAME       IMAGE                              COMMAND                  SERVICE    CREATED          STATUS
                  PORTS
broker1    apache/kafka:3.9.0                 "/__cacert_entrypoin…"   broker1    42 seconds ago   Up 39
seconds (healthy)   0.0.0.0:9092->9092/tcp, [::]:9092->9092/tcp, 0.0.0.0:9101->9101/tcp, [::]:9101->910
1/tcp
broker2    apache/kafka:3.9.0                 "/__cacert_entrypoin…"   broker2    42 seconds ago   Up 39
seconds (healthy)   0.0.0.0:9094->9094/tcp, [::]:9094->9094/tcp, 0.0.0.0:9102->9102/tcp, [::]:9102->910
2/tcp
broker3    apache/kafka:3.9.0                 "/__cacert_entrypoin…"   broker3    42 seconds ago   Up 40
seconds (healthy)   0.0.0.0:9096->9096/tcp, [::]:9096->9096/tcp, 0.0.0.0:9103->9103/tcp, [::]:9103->910
3/tcp
kafka-ui   ghcr.io/kafbat/kafka-ui:v1.1.0     "/bin/sh -c 'java --…"   kafka-ui   41 seconds ago   Up 38
seconds             0.0.0.0:8081->8080/tcp, [::]:8081->8080/tcp
```

En la siguiente imagen se puede observar la salida del paso 5:

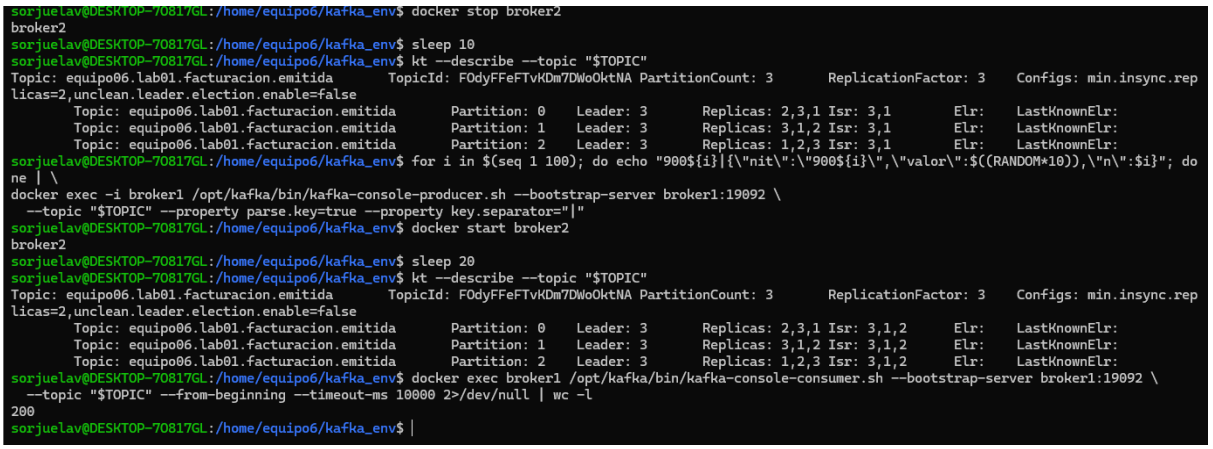
- Verificar brokers del clúster





En la siguiente imagen se puede observar además de la salida del paso 7:

- Describir líder, réplicas e ISR
- El Broker líder de la partición 0 antes de la caída era el 2
- Salida después de detener el broker, el líder ahora es el 3 pero la replicación ahora solo está en 3, 1, el 2 ya no está activo.
- Salida después de reiniciar el broker, el 3 sigue siendo líder y la replicación nuevamente tiene en cuenta las tres réplicas.
- Conteo final de mensajes, 200 porque ejecute la escritura de 100 mensajes dos veces.



1. Respuestas:
    - ¿Qué archivos componen un segmento de Kafka?
        i. archivo de datos (.log)
        ii. archivo de índice de offsets (.index)
        iii. archivo de índice de tiempo (.timeindex).
    - ¿Qué pasó con el ISR durante la caída?
        i. El broker 2 fue excluido del conjunto de réplicas sincronizadas (ISR), por lo que el ISR se redujo únicamente a los brokers activos 3 y 1.
    - ¿Por qué no se perdieron mensajes?
        i. Porque la configuración de min.insync.replicas=2 se mantuvo con las réplicas 3 y 1 (cumpliendo el mínimo requerido).
    - ¿Qué riesgo habría con `min.insync.replicas=1` y `acks=1`?
        i. El riesgo principal es la **pérdida de datos**: el líder confirma la escritura al productor tan pronto como guardará el mensaje localmente sin esperar réplicas, por lo que si el líder falla antes de replicar, los mensajes se pierden de forma permanente.